01-Análisis de Algoritmos

Diplomado Data Engineer

Universidad de Santiago de Chile

UNIVERSIDAD DE SANTIAGO DE CHILE

Juan Pablo Castillo

Funciones en Python

Breve Repaso

- Vamos a repasar que son las funciones y como se usan en Python, ya que las utilizaremos con frecuencia.
- Una función es una sección de un programa que realiza cálculos de manera independiente al resto del programa.
- Una función tiene 3 componentes principales:

  1. ***Parámetros***: valores que la función recibe como entrada.
  2. ***Código de la función***: operaciones que realiza la función
  3. ***Resultado (o valor de retorno)***: valor final que entrega la función.

- Esto nos permitirá reutilizar este mini programa en otras partes de nuestro programa principal.

Funciones

Ejemplo en Python

```python
def factorial(n):
    f = 1
    for i in range(1, n + 1):
        f *= i
    return f
```

- Las funciones en *python* comienzan con la palabra **`def`**.
- Luego, sigue el nombre que tendrá la función en este caso **`factorial`**.
- Siguen los parámetros (la entrada de la función), en este caso es uno solo: **`n`**.
- El cuerpo de la función, especifica el calculo a realizar.
- Finalmente el **`return`**, el valor a retornar por la función.

¿Que es un Algoritmo?

Conjunto de instrucciones (finito) o reglas bien definidas y ordenadas que permiten realizar una actividad (solucionar un problema) sin ambigüedad alguna.

- Note la similitud a una receta de cocina.
- Un algoritmo tiene una entrada de cierto tamaño (número de elementos a procesar)
  - Generalmente le diremos n a este dato.
- El algoritmo debe producir una salida luego de completar su ejecución.
- El ejemplo anterior de la función `factorial` era un algoritmo que resuelve el problema de calcular el factorial dado un número `n` dado.

Ejemplo de un Algoritmo

```
def promedio(lista):
    suma = 0.0
    n = len(lista)
    if n == 0:
        return
    for numero in lista:
        suma += numero
    return suma/n
```

En este caso n es el tamaño de la lista a calcular el promedio

Motivación

- Dado un algoritmo/programa determinado, nuestro objetivo es determinar la "*calidad*" nuestro algoritmo.
- Es decir, determinar que tan "*rápido*" o "*lento*" es.
- Es sumamente importante esto cuando trabajamos con un volumen de datos significativo ("Big Data").
- ¿Pero como analizamos nuestros programas?
  - Probablemente nuestra primera idea sería ejecutarlo varias veces para distintas entradas y registrar su tiempo de ejecución (e.g., en segundos, ms, ns, etc).

Motivación

- ¿Que problemas podríamos tener usando esta idea?

1) **Que pasa si ejecutamos nuestro programa en una maquina diferente?**

    - Tendríamos resultados distintos, probablemente cada uno de nosotros.

2) **Y si nos damos cuenta que es “malo” nuestro algoritmo después de implementarlo/programarlo?**

    - Perdemos tiempo y recursos implementando la solución.

3) **Como escogemos las entradas a probar?**

Motivación

- Lo mejor que podemos hacer es abstraernos de **donde** correremos nuestro programa y del **lenguaje de programación.**
- Para eso usaremos ***notación asintótica.***
- Son técnicas de estimación del costo de un algoritmo.
- Mediremos la eficiencia de un algoritmo para entradas “muy grandes” (ya que estamos pensando en manipular grandes cantidades de datos).
- Esto nos permitirá entender los costos de un algoritmo o programa **incluso antes de implementarlo.**

Notación Asintótica

Pero, ¿Como lo hacemos?

<u>Tiempo de Ejecución de un Algoritmo:</u>

- Lo calcularemos contando la cantidad de operaciones básicas ejecutadas como función del tamaño de la entrada. La llamaremos $T(n)$.
  - Note que, n puede ser cualquier valor (positivo) y solo nos estamos abstrayendo de darle algún valor en particular.
  - Ya no necesitamos saber donde y en que lenguaje se ejecutará nuestro programa.

Operaciones Básicas

Operaciones	Sintaxis en Python
Asignación	+=, *=, =
Aritméticas	+, *, -, +, /, //
Lógicas	and, or, not
Relacionales	<=, >=, <, >, !=, ==
Saltos	return, break

Tiempo de Ejecución

Reglas para calcular tiempos de ejecución:

Expresión	Tiempo de ejecución
Operaciones Básicas	1 unidad de tiempo
Secuencia de sentencias	Suma de los tiempos de cada sentencia.
if (cond): **S1** **else:** **S2**	T(cond) + max{T(S1), T(S2)}
while(cond): **S**	T(cond) + #iteraciones * (T(S)+T(cond))

- Esto nos dice que seremos **pesimistas** a la hora de calcular el tiempo de ejecución.
- Es decir, pensaremos siempre en el mayor tiempo de ejecución que podría ocurrir en nuestro algoritmo/programa.

Notación Asintótica

- El tiempo de ejecución de nuestro ejemplo anterior?
  - n iteraciones (la cantidad de datos de la lista)
  - Cada iteración tiene un costo de c operaciones básicas.
  - El tiempo sería $T(n) = c \cdot n$, es decir, crece linealmente respecto al tamaño de la entrada.

Notación Asintótica

Un poco más de detalle

```
def promedio(lista):
    suma = 0.0
    n = len(lista)
    if n == 0:
        return
    for numero in lista:
        suma += numero
    return suma/n
```

Entrada de tamaño n

Bucle `for` que itera sobre los n datos de la

- Note que hay múltiples operaciones básicas, si las contamos:

$$T(n) = 2 \cdot n + 7$$

- Para un n "*muy grande*" las constantes 2 y 7 serán despreciables, es decir, $T(n) \approx c \cdot n$
- **El tiempo crece linealmente a la entrada!**

Otro Ejemplo

```
def maximo(lista):
    n = len(lista)
    if n == 0:
        return
    maximo = lista[0]
    for i in range(1, n):
        if lista[i] >  maximo:
            maximo = lista[i]
    return maximo
```

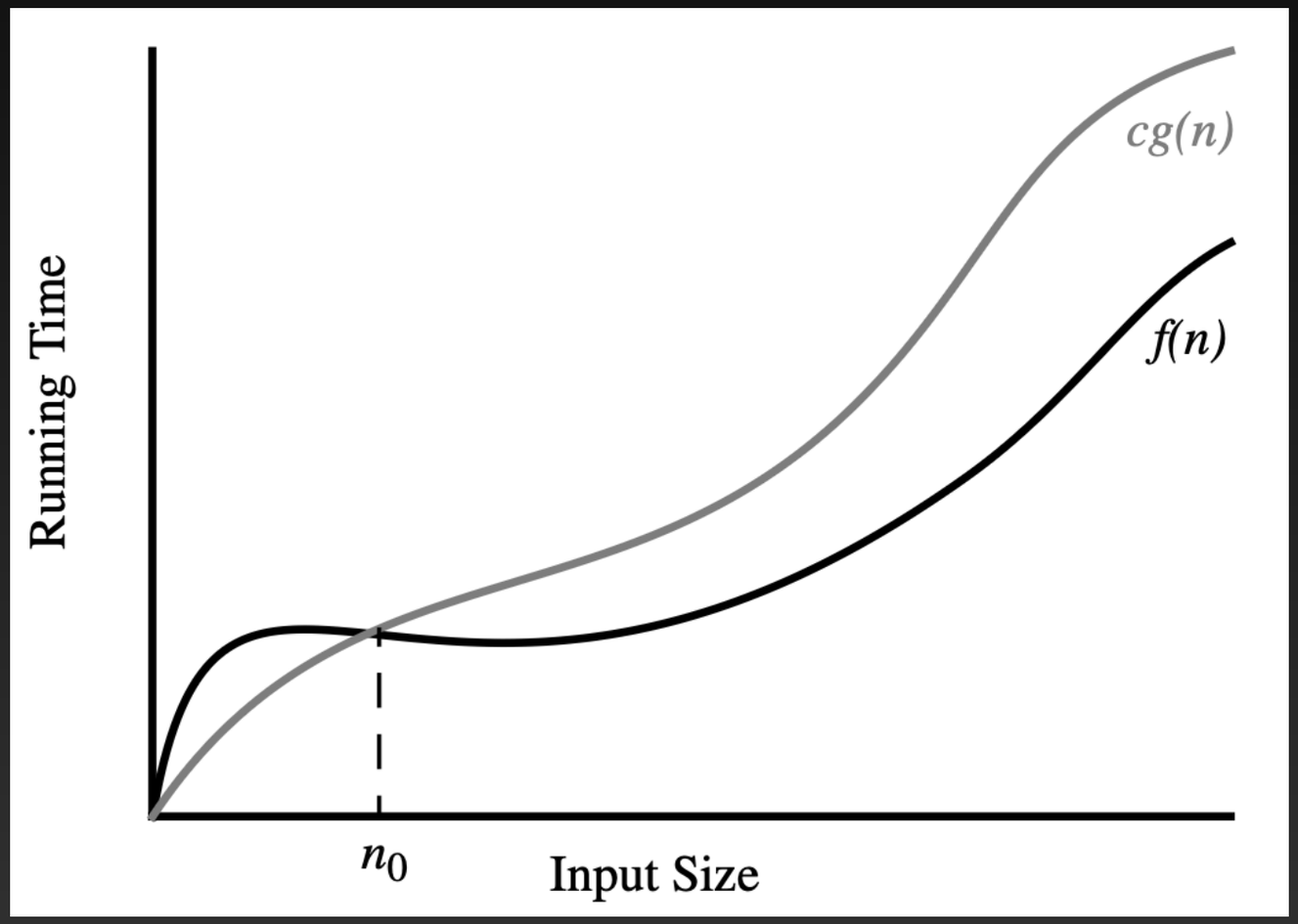
- ¿Cual sería su tiempo de ejecución?

$$T(n) = c \cdot n$$

  - Asigna el primer elemento como el máximo.
  - Luego itera sobre los $n - 1$ datos restantes, buscando si hay un dato más grande.

Notación *"O" grande*

- La idea es acotar nuestro tiempo de ejecución $f(n) = T(n)$, con alguna otra función $g(n)$.



Para esto, $g(n)$ debe cumplir que:

$$f(n) \leq c \cdot g(n)$$

para algún n_0 dado.

En resumen, debemos **acotar el tiempo con alguna función** $g(n)$**,** que crezca más rápido que $f(n)$ desde un n inicial dado.

Notación *"O" grande*

Veamos un ejemplo en concreto

```
def maximo(lista):
    n = len(lista)
    if n == 0:
        return
    maximo = lista[0]
    for i in range(1, n):
        if lista[i] >  maximo:
            maximo = lista[i]
    return maximo
```

- Es el mismo caso que habíamos visto anteriormente, y concluimos que $T(n) = c \cdot n$.
  - Se comporta de forma lineal respecto a la entrada.
- Entonces, podemos decir que $T(n)$ es $O(n)$.

Notación *"O" grande*

En concreto, podemos decir que cualquier función lineal es , y uno lo pudiera generalizar de la siguiente forma:

- $f(n) = 15 \in O(1)$, "**funciones" constantes.**
- $f(n) = 5n^3 + 2n - 3 \in O(n^3)$, **funciones cúbicas.**
- $f(n) = \dfrac{n^2}{2} + 1 \in O(n^2)$, **funciones cuadráticas.**
- $f(n) = 3n - 1 \in O(n)$, **funciones lineales**
- $f(n) = \log_2 n - 1 \in O(\log_2 n)$, **funciones logarítmicas**.
- Etc...

Notación *"O" grande*

El proposito de esto?

- Es más fácil decir que algoritmo es mejor sin preocuparse demasiado en el detalle
  - Es decir, fácil de comparar.
- Es mucho más sencillo de leer.
- Se entiende el concepto o se da por entendido, que se están contando las operaciones elementales.
- En la mayoría de documentaciones se entrega esta información, para indicar la eficiencia de la función/método que se está utilizando.

Notación *"O" grande*

Algunas Reglas que Podemos Seguir

Regla de la suma

Si $f_1 \in O(g)$ y $f_2 \in O(h)$, entonces:

$$f_1 + f_2 \in O(\max\{g, h\})$$

Por ejemplo:

Sea

$f_1 = \dfrac{n^2}{2} \in O(n^2)$ y $f_2 = 2n - 1 \in O(n)$, entonces:

$$f_1 + f_2 = \frac{n^2}{2} + 2n - 1 \in O(\max\{n^2, n\})$$

Notación *"O" grande*

Algunas Reglas que podemos seguir

Regla del producto

Si $f_1 \in O(g)$ y $f_2 \in O(h)$, entonces:

$$f_1 \cdot f_2 \in O(g \cdot h)$$

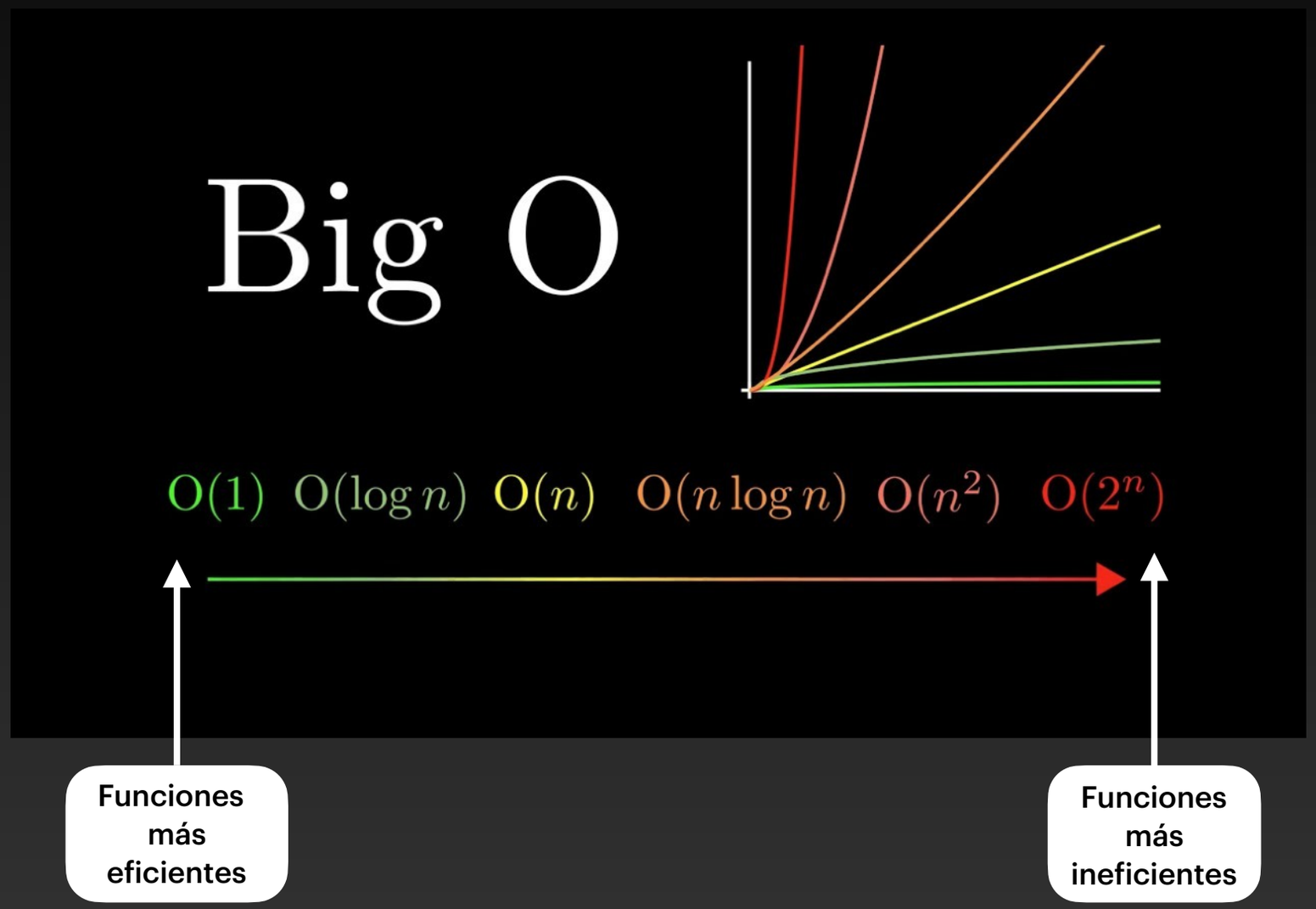
Por ejemplo:

Sea $f_1 = \dfrac{n^2}{2} \in O(n^2)$ y $f_2 = 3n \in O(n)$, entonces:

$$f_1 \cdot f_2 = \frac{n^2}{2} \cdot 3n \in O(n^2 \cdot n) = O(n^3)$$

Notación “O” grande

Algunas de las funciones más importantes:



Está gráfica nos sirve para rápidamente comparar que tan eficientes son nuestros algoritmos/programas.

Mejores Máquinas o Mejores Algoritmos?

Funciones vs #operaciones

n	log n	n	n log n	n^2	n^3	2^n
8	3	8	24	64	512	256
16	4	16	64	256	4096	65536
32	5	32	160	1024	32768	4294967296
64	6	64	384	4096	262144	1.84×10^{19}
128	7	128	896	16384	2097152	3.40×10^{38}
256	8	256	2048	65536	16777216	1.15×10^{77}
512	9	512	4608	262144	134217728	1.34×10^{154}

- Note como a medida que la función es **más ineficiente**, la cantidad de operaciones crece abruptamente.
- Imagine que tenemos una cpu el doble de potente (es decir, es capaz de realizar más operaciones básicas por segundo).
  - Dependiendo de la función que represente al tiempo de ejecución veremos las mejoras.
  - Pero note que para la función 2^n no importa que tan potente sea nuestra cpu, las mejoras serán **insignificantes.**

Actividad

Dado el siguiente algoritmo, responda las preguntas.

```python
from sys import maxsize # número muy grande

def sort(lista):
    n = len(lista) # largo de la lista
    listaOrdenada = []
    for i in range(0, n): # de i=0 a i=n-1
        min = maxsize
        minIndex = 0
        for j in range(0, n): # de j=0 a j=n-1
            if lista[j] != "x" and lista[j] < min:
                min = lista[j]
                minIndex = j
        lista[minIndex] = "x"
        listaOrdenada.append(min)

    return listaOrdenada
# Ejemplo de utilización de la función sort
lista = [2, 5, 1, 4, 6, 10, 3]
print(sort(lista))
```

Preguntas en la siguiente ppt

Actividad: Preguntas

- **Explique brevemente qué hace el algoritmo.**
  - No es necesario describir línea a línea, explique a grandes rasgos qué hace el algoritmo para cumplir su objetivo.
    - Ojo, la idea es ser más especifico que decir "el algoritmo ordena".
- **Cuál es el tiempo de ejecución del algoritmo?**
  - Puede asumir que el operador `append()` toma tiempo $O(1)$.
  - Justifique su respuesta con los contenidos vistos.
- **¿Cuál sería el tiempo de ejecución del algoritmo usando la notación "*O*" grande?**
  - Justifique su respuesta con los contenidos vistos.

Reglas de Entrega

- Debe entregar su tarea en la entrega habilitada en uvirtual.
- Puede adjuntar un archivo pdf con sus respuesta en su entrega o escribirlas directamente el .ipynb
- La fecha limite de la entrega es el día miércoles 15 de abril a las 18:59hrs*.